1A:

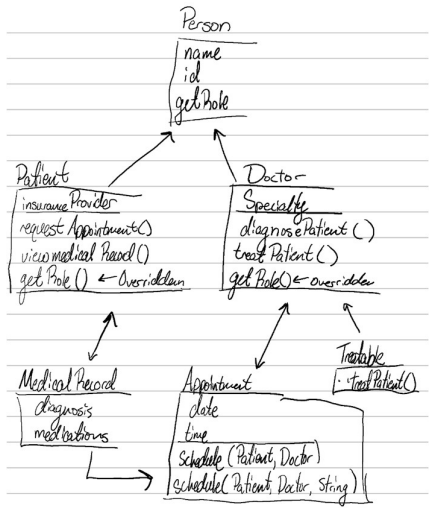
- **Person (base class)**
- **Patient** → Person (inherits), has MedicalRecord
- **Doctor** → Person (inherits), implements Treatable
- **MedicalRecord** → belongs to Patient (composition)
- **Appointment** → links Patient and Doctor
- **Treatable (interface)** → implemented by Doctor

Concepts

- Inheritance
  - Patient and Doctor class inherit from person, reusing shared attributes name and id
- Encapsulation
  - Attributes like medicalRecords are private; access can only be gained by invoking viewmedicalrecord()
- Interfaces
  - Treatable interface defines treatPatient. Which only a doctor may implement to ensure consistent treatment behavior
- Method Overloading
  - Appointment.schedule() is overloaded with accepting (patient, Doctor) and (patient, doctor, String) for notes.
- Method Overriding
  - getRole() in Patient and Doctor override Person.getRole() to return the correct role.
- Composition
  - Patient has a medicalRecord and Appointment has a reference to both patient and doctor.

1B:

Figure 1. UML



2A: Code turned in alongside PDF

2B:

1. Encapsulate what varies
    a. This system puts the sales tax logic in different locations instead of putting all the logic into one class. Each tax rule has its own strategy class to be called when a state is created.
2. Favor composition over inheritance
    a. The state class does not inherit tax logic. Instead, it contains the SalesTaxBehavior object that perfomes the tax calculation. This composition allows varying tax behaviors to be assigned to states rather than hard coded. Allows flexibility.
3. Program to an interface, not an implementation
    a. The state class depends on the SalesTaxBehavior interface rather than a concrete class like Tax Strategy classes. This means new tax strategies can be added without modifying the state class.

2C: Create an abstract super class (tax Strategy) or interface which will take the tax logic and make them concrete subclasses. Thus, making the main program refers to tax strategy instead of the classes.

3:

1. IS-A
    a. Mallard inherits from Duck, representing a subtype.
2. IS-A
    a. RedheadDuck inherits from Duck.
3. IS-A
    a. RubberDuck inherits from Duck.
4. IS-A
    a. DecoyDuck inherits from Duck
5. HAS-A
    a. Duck contains a FlyBehavior object.
6. HAS-A
    a. Duck contains a QuackBehavior object.
7. IS-A
    a. Quack inherits from QuackBehavior, representing a subtype.
8. IS-A
    a. Squeak inherits from QuackBehavior.
9. IS-A
    a. MuteQuack inherits from QuackBehavior.